SEASUG 2026 Hand-On Workshop

CODING FOR CLEANLINESS IN SAS® VIYA®

Mark Jordan, SAS Jedi (Retired)

ABSTRACT

Cleaning messy data consumes about 80% of most programming projects. SAS Viya's Data Quality tools like the Quality Knowledge Base (QKB) are often underused by experienced SAS programmers and offer language- and geography-specific tools for quickly and easily standardizing text and precision fuzzy matching.
This code-forward session shows you how the QKB works and how to choose the right locale, standardize inconsistent values, and generate match codes for clustering, de-duplication, and fuzzy joins in SAS. You'll leave with ready-to-run example code for your own data.

INTRODUCTION

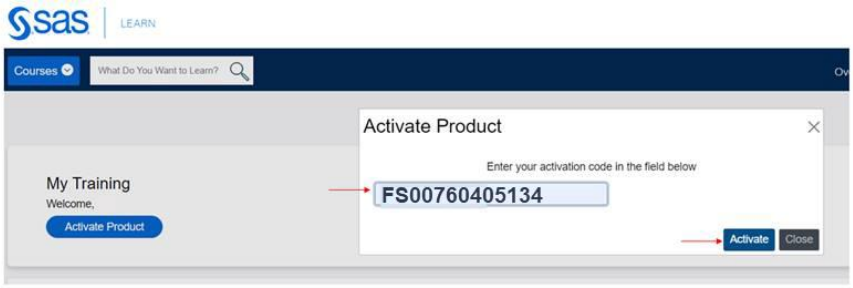
In this workshop I'll discuss and demonstrate:

- Data Quality technology available in SAS Viya
- The anatomy of a Quality Knowledge Base (QKB)
- Languages and Locales
- Identifying the available QKBs and Locales
- Generating match codes with Data Quality procedures and functions to
- Clustering and de-duplicating data
- Standardizing data values
- Performing fuzzy joins for data enrichment

LAB SETUP

ACCESSING VIYA FOR LEARNERS

1. Sign into your SAS Profile at: **https://www.sas.com/profile/**
   If you don't have a SAS Profile, you must create one (it's free).
2. Access your 'My Training' page at: **https://learn.sas.com/my/**
   NOTE: If you use an activation code provided by the conference session leader, it will expire when the conference is over. If already you have access to VFL, use your own access instead.
3. To use the conference code, go to the My Training' page, input the code in the Activation Code box, and click "Activate":

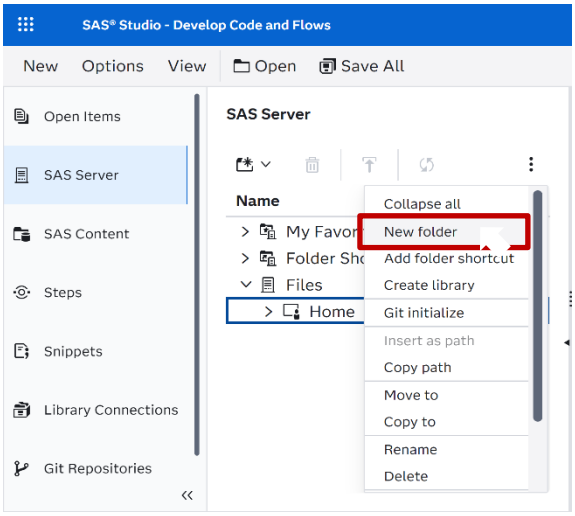


4. Accept/Submit the SAS 'My Training' Licensing Agreement
5. Go to the SAS Viya for Learners Launch Page: https://learn.sas.com/vfl
   Use the SAS VFL Launch Page each time you access SAS Viya.
6. Click the 'Launch' button to Launch VFL

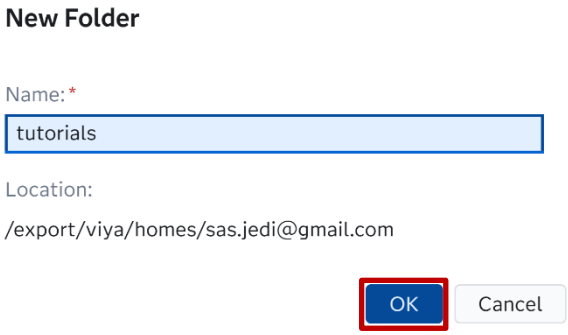
Coding for Cleanliness in SAS® Viya®

SET UP FOR EXPLORATION AND PRACTICE

1. Log in to sas viya for learners and navigate to sas studio (develop code and flows)
2. In the Explorer, choose SAS Server, right-click on Home, and choose New folder.



3. Name the new folder "tutorials" – all letters lower case (CASE MATTERS) then click "OK"



Set up for Exploration and Practice

4. Write and execute this code (URL is CASE Sensitive!):

```
filename setup url "https://bit.ly/getC4Cdata";
 %include setup;
 %SETUPCODE4CLEANLINESS(~/tutorials)
```

5. In the Explorer, open and execute program **`setup.sas`** from folder
   `~/tutorials/coding for cleanliness`

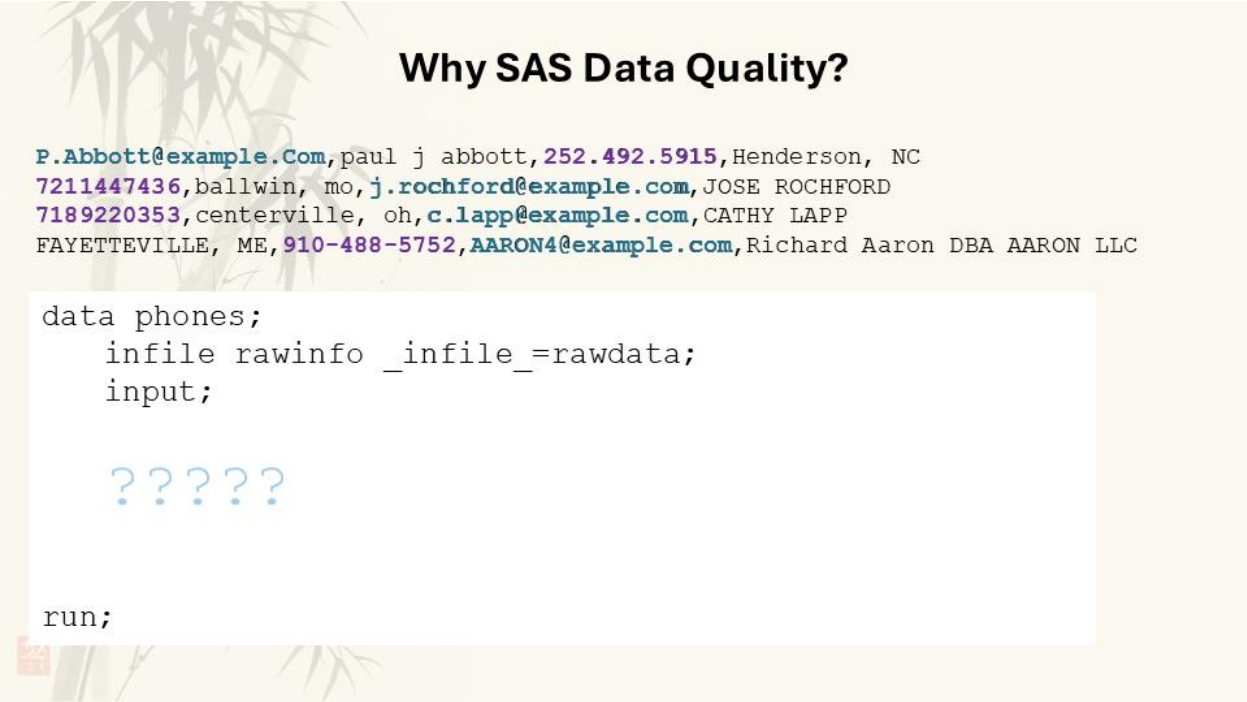
6. This report indicates success:

Tables in SASDATA and CASUSER libraries

SASDATA	CASUSER
CONTACTS	CONTACTS
EXAMPLES	EXAMPLES
FORMATS	FORMATS
NAMES	NAMES
PROSPECTS	PROSPECTS
RAWCONTACTS	RAWCONTACTS
RAWINFO	RAWINFO

INTRODUCTION TO SAS DATA QUALITY

WHY SAS DATA QUALITY?



By now, you know the 80/20 rule by heart: 80% of your time on any data-driven project is spent getting the data into usable shape, and the remaining 20% is spent doing the work you actually intended to do. Sometimes your data comes in text files like this, created by applications over which you have no control. The data you need is in there, but how will you get it out? I see that there is an @ sign in every email, and a comma comes before and after – I can work with that. But what if I also need the phone numbers? How am I going to code for that?

RowID	Contact	Address	City	State	ZIP
1	Mr. John Q. Smith	719 Madison Ave	Cary	NC	27513
2	Johnny Smythe	719 Madison Avenue	Cary	North Carolina	23185
… lots more rows!					
3000000	smith, jon q.	719 madisun av.	cary	nc	27513

Even with structured data, values are not always consistently entered. How can you tell if there is more than one entry for the same individual? Sure, it's easy to "eyeball" it with 2 rows of data, but that's not a solution when there are millions of rows instead – expecially when the data is VERY dirty.

THE SAS QUALITY KNOWLEDGE BASE (QKB)

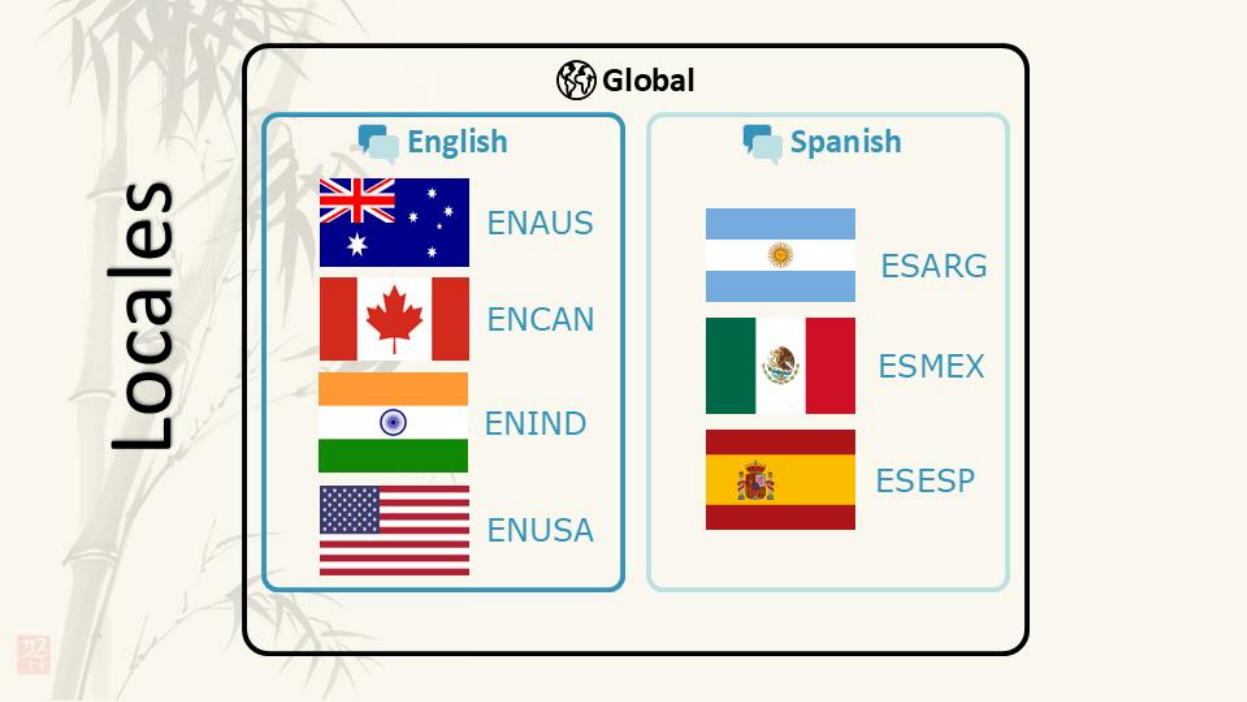


All SAS Viya implementations include SAS Data Quality, and central to its workings is the Quality Knowledge Base (QKB). Each SAS Data Quality installation includes a QKB called the SAS Quality

Knowledge Base for Contact Information, which supports a vast array of SAS processes used to profile and modify your data in a variety of ways. The QKB contains 10 definition types, inmore than 30 languages with multiple locale variations, and about 1900 individual definitions that can be leveraged by multiple locales. Several procedures, functions, and autocall macros are provided to leverage QKB functionality in your SAS code and applications.

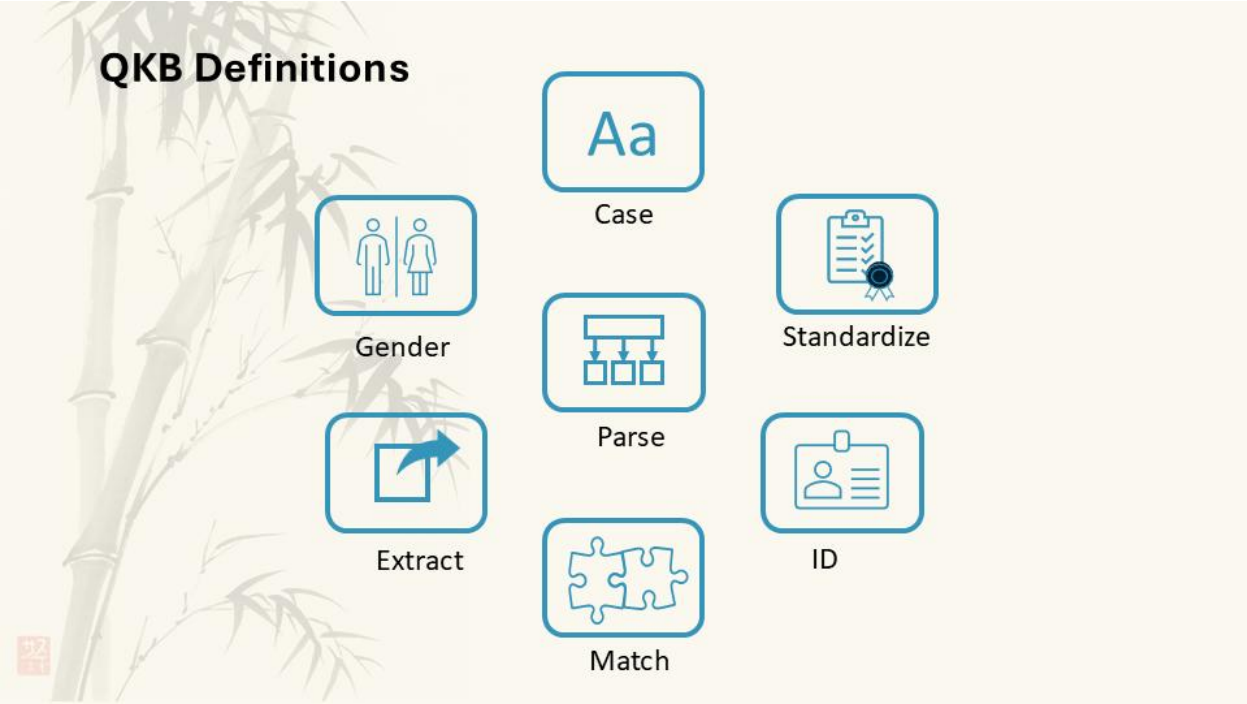


The definitions in the QKB are specific to a geography, a language, or to both. The combination of geography and language is known as a locale. For example, a definition might be associated with English as spoken in the United States – the ENUSA locale, or English as spoken in Australia – the ENAUS locale. You need to know your data's locale so you can get the best results when referencing QBK definitions.

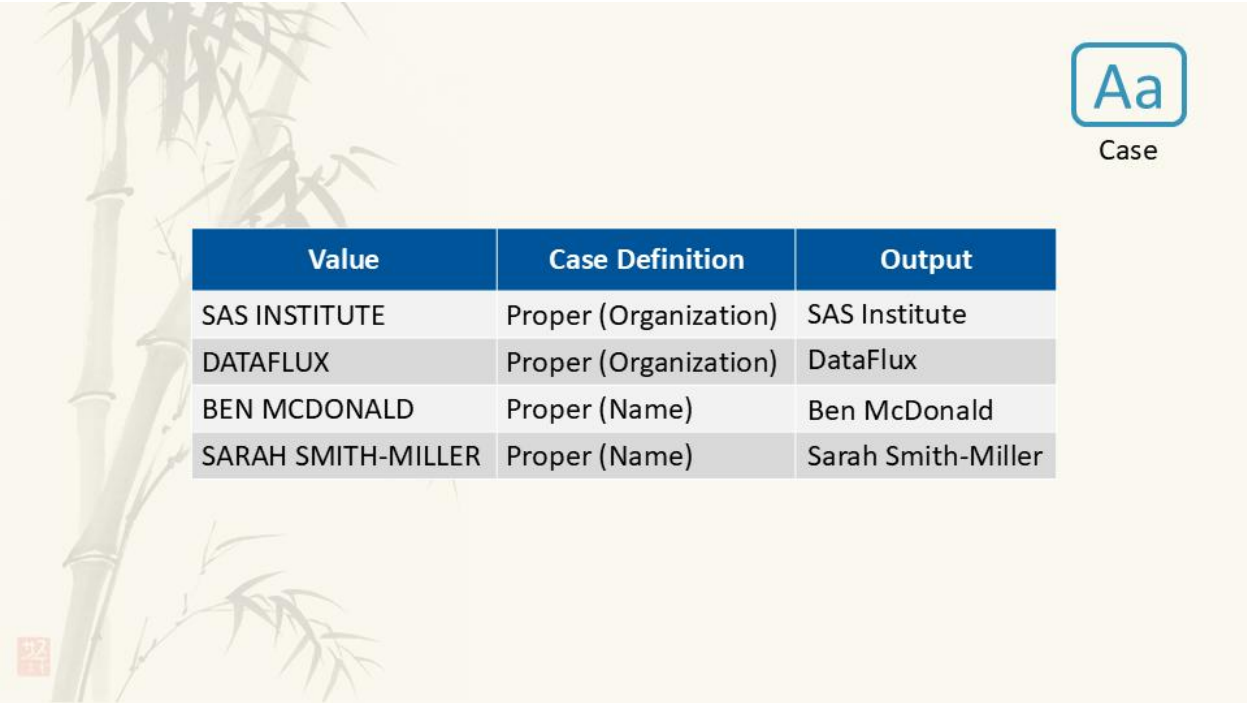


QKB locales are hierarchical. The three levels of hierarchy are: global, language, and locale, with each level inheriting data and definitions from its parent level. The global level contains definitions and data types that are always applicable, regardless of language or locale. At the language level are the data and definitions specific to a language, like English and Spanish. And finally, locales contain data and definitions specific to a language as spoken in a particular geographical region. For example, the English language might include locales for Australia, Canada, India, and the United States, while the Spanish language might include locales for Argentina, Mexico, and Spain.

QKB DEFINITIONS



A QKB definition is a set of steps for processing data values, and there are a wide variety of definition types available, including Parse, Case, Standardization, Identification Analysis, Match, Extraction, and Gender Analysis definitions. We won't have time to cover all of these today.

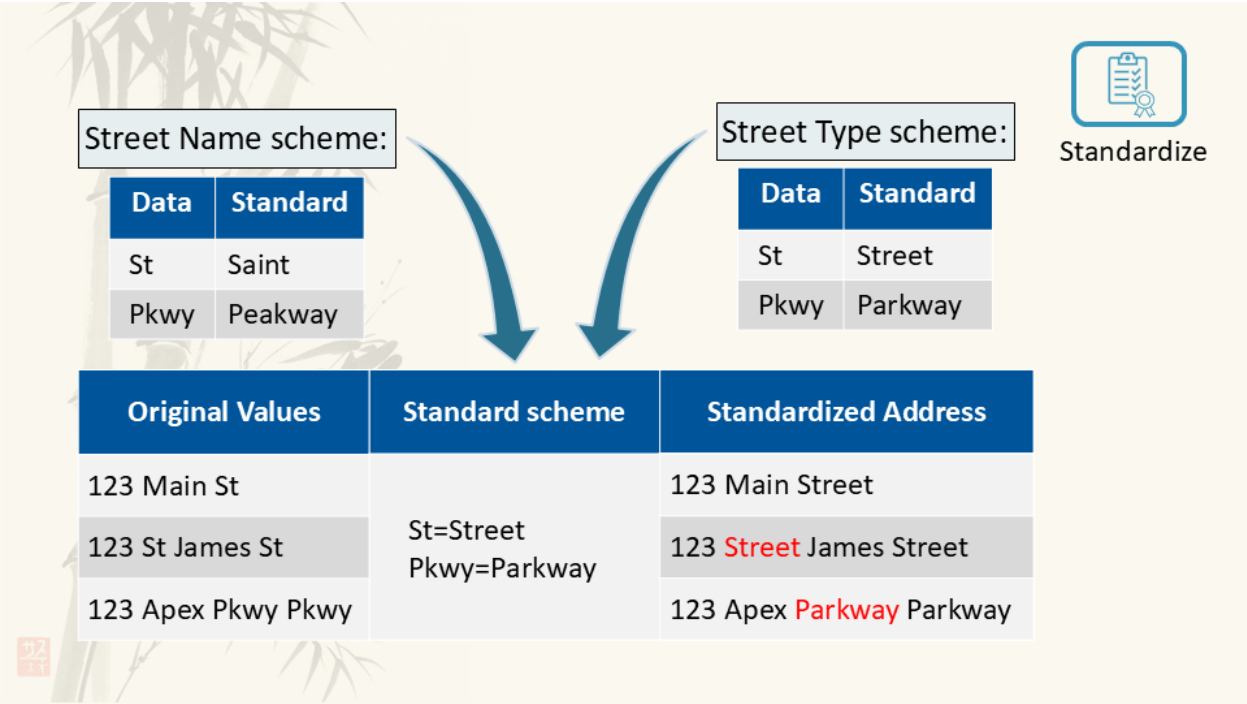


Value	Case Definition	Output
SAS INSTITUTE	Proper (Organization)	SAS Institute
DATAFLUX	Proper (Organization)	DataFlux
BEN MCDONALD	Proper (Name)	Ben McDonald
SARAH SMITH-MILLER	Proper (Name)	Sarah Smith-Miller

We will leverage case definitions to apply appropriate letter casing to a character string. Case definitions are especially useful for words with specialized casing requirements, for example, company names and surnames.



Parse definitions are used to divide a string into the tokens appropriate for the data type. For example, this string contains the text "John.Smith@sas.com". The E-mail parse definition breaks this string down into mailbox, sub-domain, and top-level domain tokens. After parsing, the mailbox token contains the text "John.Smith", the sub-domain value is "sas", and the top-level domain is "com".



Standardization definitions standardize the appearance of text values based on the content, capitalizing words appropriately, and removing adjacent blank spaces and unnecessary

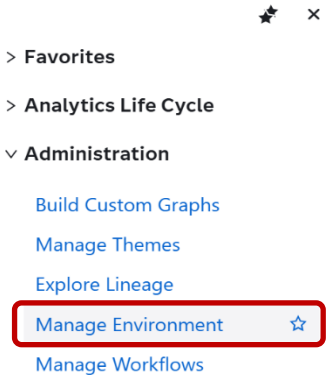
punctuation. Applying the same standardization scheme to every word in a string often yields unsatisfactory results. Standardization definitions parse a string into tokens and then apply one or more schemes to each token. For example, an address standardization scheme might use the Street Name scheme to standardize the first occurrence of a token and the Street Type scheme for the second occurrence, producing much more desirable results.

EXPLORING SAS DATA QUALITY IN MY SAS SESSION

Explore 1 – Which QKB is available?

Let's explore our environment a litte bit, with an eye towards understanding the SAS Data Quality tools available in our SAS Session.
Follow these steps:

1. Log in to SAS Viya
2. Select the Applications Menu (- upper left)
3. Choose "Manage Environment"



4. What is the name of the installed QKB?
5. Go to SAS Studio (Develop code and flows).
6. Open and execute program "e01 - List QKBs.sas" and run the code in Step 1. Look in the Log – is QKB CI 33 available? **YES** **NO**
7. Run the code in Step 2. What is the default locale for this session? **PTBRA ENUSE ESMEX**

Explore 2 – Investigating Locales

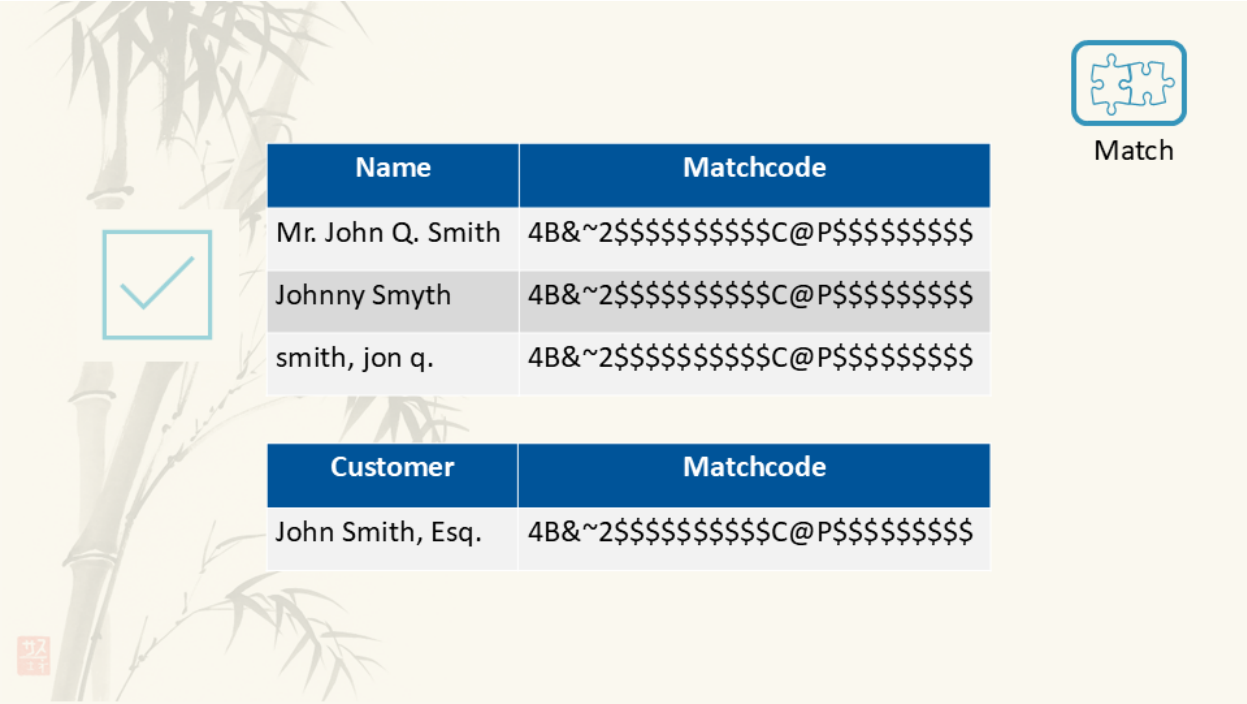
In this exploration, we'll use the DQLOCALEINFOGET function to check which locales are loaded for our SAS session, then use the %DQLOAD macro to load an additional locale. Finally, we'll explore the definitions available in those locales using the DQLOCALEINFOLIST function. Go to SAS Studio (Develop code and flows), and open program e02 – Explore QKB.sas

1. Run step 1. What locales are loaded for this session?

2. Run the code in Step 2 to load an additional locale, then re-run the code in step 1. What locales are loaded now?

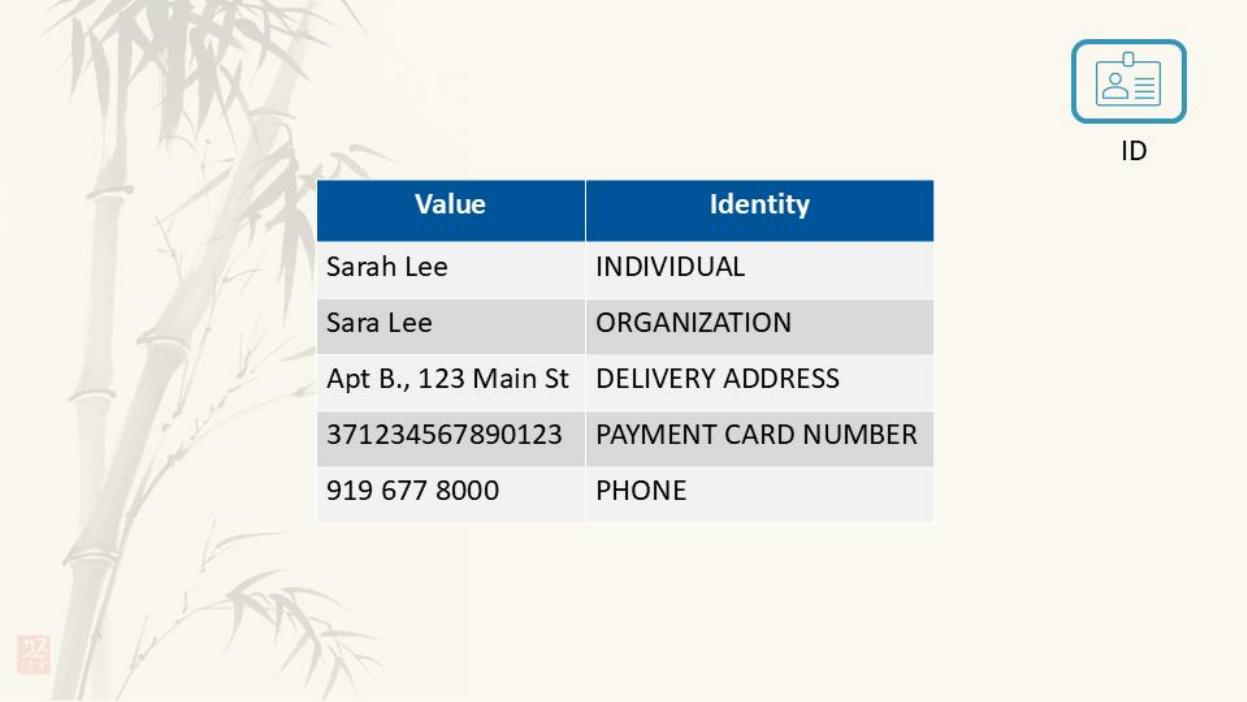
3. Run the code in Step 3.
   How many case definitions are available for the ENUSA locale?
   How many case definitions are available for the PTBRA locale?

MATCH CODES



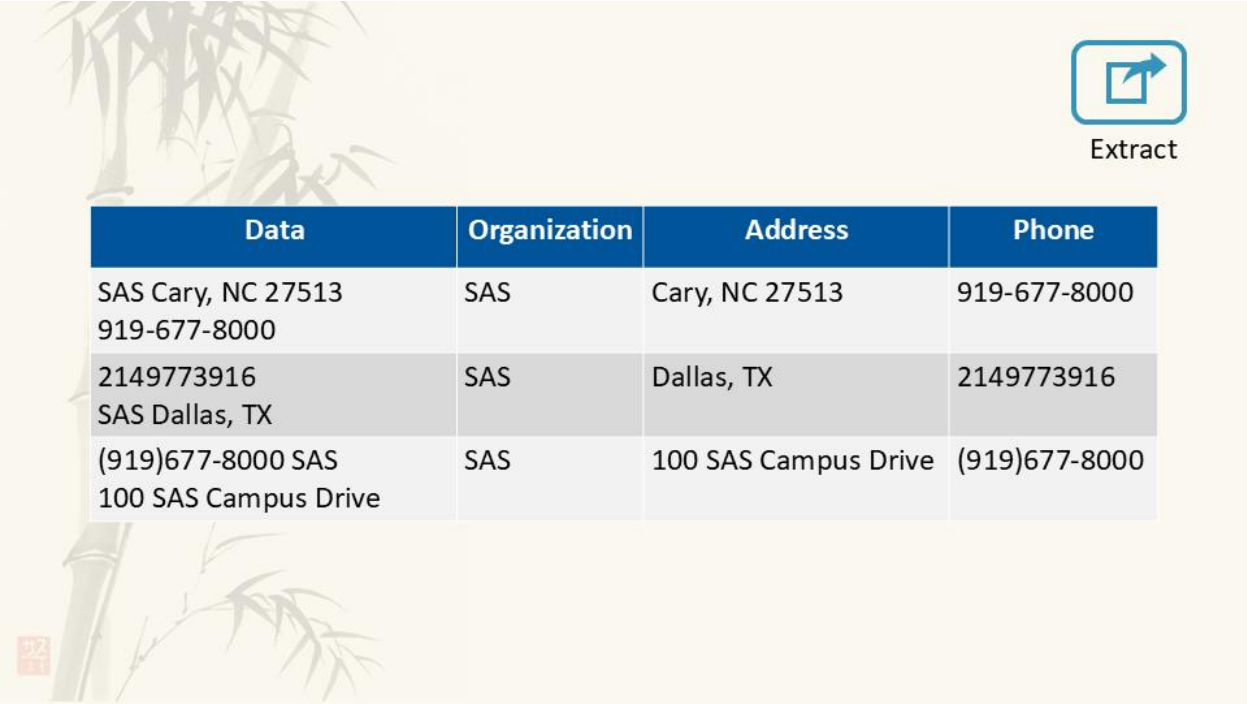
Matchcodes are a fuzzy representation of a data value. SAS Data quality can use matchcodes to make fuzzy matching magically easy to code. Consider the names "Mr. John Q. Smith", "Johnny Smyth", and "smith, jon q.". Most humans would conclude that these records potentially refer to the same individual, but a computer using standard matching techniques would never identify these as a match. The matchcodes generated for these records make it easy to group them together within a table or perform "fuzzy matching" to data in other tables where a common unique key is not available.

IDENTIFICATION ANALYSIS



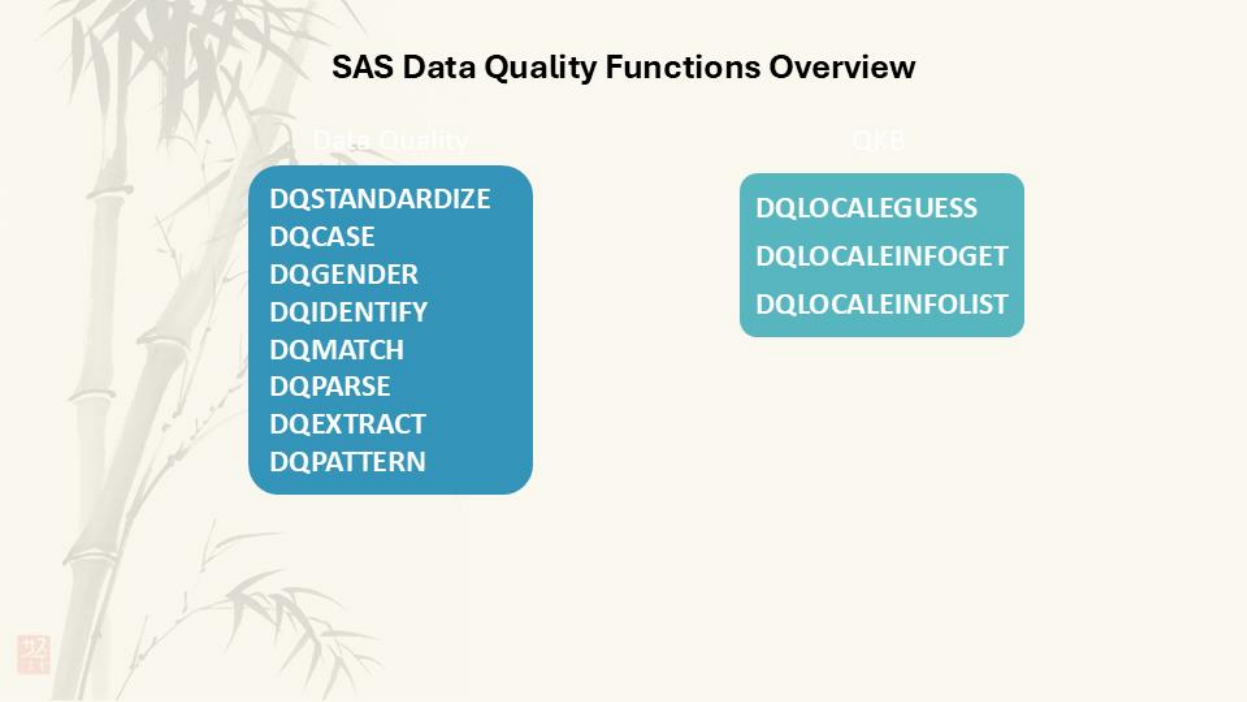
Identification analysis definitions can be used to identify the data types of a text value. For example, the string *Sarah Lee* is identified as an *INDIVIDUAL*, but *Sara Lee* is a trademark, so is identified as an *ORGANIZATION*. Other data types can be identified by this definition as well.

EXTRACTION DEFINITIONS



An extraction definition can be used to extract components from a string. For example, an extraction definition can separate complex address data into individual address components, such as organization, address, and phone. The definition also recognizes name, e-mail, and additional information tokens. The order of the components in the original text doesn't matter.

SAS DATA QUALITY FUNCTIONS OVERVIEW



SAS Data Quality provides a variety of functions that can perform data quality tasks, such as standardizing or casing data, or produce information about the data's locale.
Data quality functions enable you to perform a variety of data-cleansing tasks. Many data quality functions are available, but today we'll concentrate on the four used to

- perform identification analysis on a data value,
- properly case text,
- standardize data values,
- generate match codes for fuzzy matching

SELECTED DQ FUNCTIONS

DQIDENTIFY

DQIDENTIFY(*source-text*,'*identification-analysis-definition*'<, *locale*> **);**

```
proc FedSQL;
select Contact
      ,propcase (Contact) as Propcase
      ,dqidentify(Contact,'Field Content') as Identity
   from sasdata.rawContacts
   limit 5;
quit;
```

CONTACT	PROPCASE	IDENTITY
jennifer inscoe	Jennifer Inscoe	INDIVIDUAL
Trw Space & Defense	Trw Space & Defense	ORGANIZATION
DENISE MCNATH	Denise Mcnath	INDIVIDUAL
Fidelity Federal Bank	Fidelity Federal Bank	ORGANIZATION
marina tennis-smith	Marina Tennis-Smith	INDIVIDUAL

The DQIDENTIFY function applies an *identification-analysis-definition* to the *source-text* to identify the type of data represented. It has two required arguments:

- *Source-text* specifies a character constant, variable, or an expression that provides the input text to be identified.
- *Identification-analysis-definition* identifies the QKB definition used to classify source text.

An optional argument specifies the QKB *locale* containing the identification analysis definition. When locale is not specified, a default locale is used.

In this example, the **sasdata.rawContents** table contains improperly cased values in the Contact column. Applying the PROPCASE function to the data produces unsatisfactory results. To properly case the values, we first need to identify the type of entity represented.

DQCASE

DQCASE(*source-text, case-definition'<, locale>*);

```
proc FedSQL;
select Contact
   ,Propcase (Contact) as Propcase
   ,case(dqidentify(Contact,'Field Content'))
      when 'INDIVIDUAL' then dqcase(Contact, 'Proper (Name)')
      when 'ORGANIZATION' then dqcase(Contact,
                                      'Proper (Organization)')
      else propcase(contact)
     end as ContactCased
   from sasdata.rawContacts
   limit 5;
quit;
```

CONTACT	PROPCASE	CONTACTCASED
jennifer inscoe	Jennifer Inscoe	Jennifer Inscoe
Trw Space & Defense	Trw Space & Defense	TRW Space & Defense
DENISE MCNATH	Denise Mcnath	Denise McNath
Fidelity Federal Bank	Fidelity Federal Bank	Fidelity Federal Bank
marina tennis-smith	Marina Tennis-Smith	Marina Tennis-Smith

The DQCASE function applies a *case-definition* to the *source-text* to properly case the value based on the type of entity represented. The two required arguments are:

- Source-text, which specifies the text to be cased and
- *Case-definition*, which identifies the QKB definition used to case the source text.

Once again, you can specify a *locale* if desired.

In this example, the **sasdata.rawContents** table contains improperly cased values in the Contact column. We've seen that the PROPCASE function yields unsatisfactory results. But we can use the DQIDENTIFY function to identify the type of entity represented, and then use the DQCASE function to apply the appropriate case definition to get the results we desire.

DQSTANDARDIZE

DQSTANDARDIZE (*source-text*, '*standardization-definition*' <, *locale*>);

```
proc sql;
select Contact 'Name', City, State
   ,dqstandardize(State,'State/Province (Abbreviation)')as ST
    from sasdata.prospects
    where calculated ST='NY'
;
quit;
```

Name	CITY	STATE	ST
Bobby Walton	Kings	N.Y.	NY
Cindy Endrusick	Erin	NY	NY
Joe Lamanna	NY	New York	NY
Donny Kuczynski	Queens	NY	NY
Bob Inghram	Kings	N. York	NY

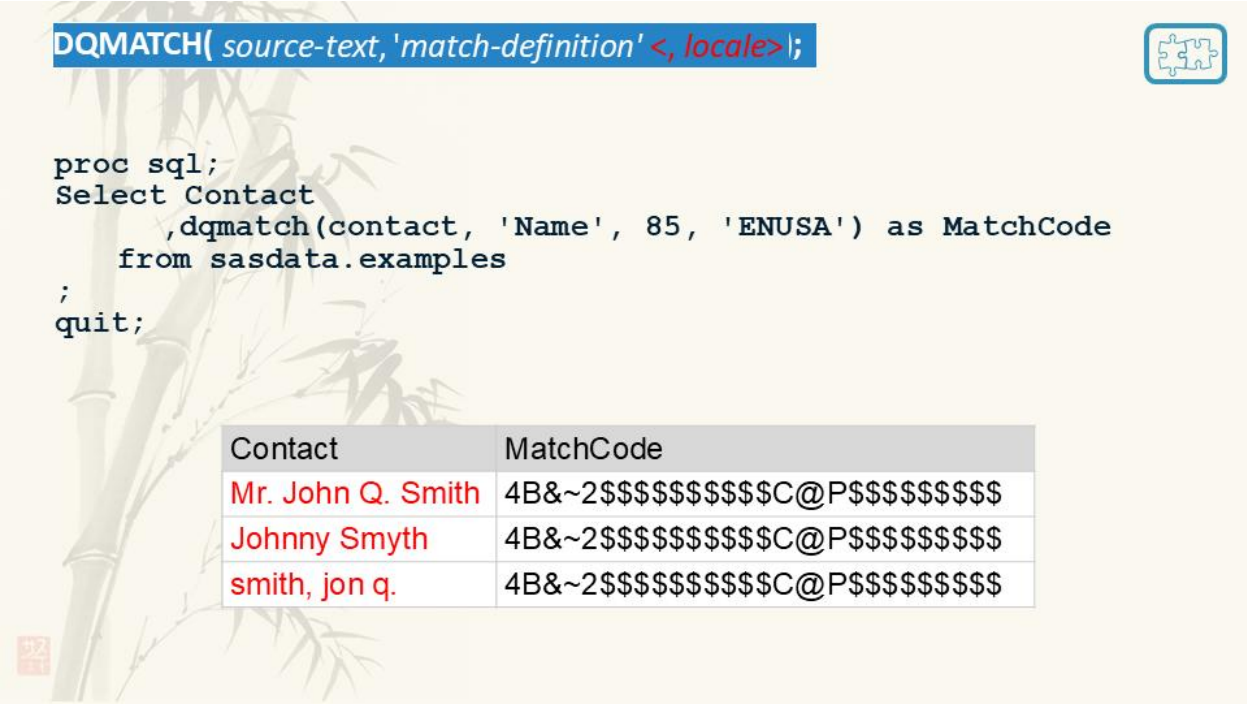
The DQSTANDARDIZE function applies a standardization definition for transforming an input value. The required are:

- *source-text,* which specifies the input text to be standardized, and
- *Standardization-definition*, which identifies the QKB standardization definition used to standardize the input text.

Again, you can specify the *locale* if desired.

In this example, the **sasdata.prospects** table contains several rows for prospects living in New York state. But because the data values are not standardized, it would be very difficult to create a report containing only prospects from New York. For visibility, I'll add a computed column named 'ST' to the report that uses the DQSTANDARDIZE function to standardized the values in the State column. By using the standardized value in the WHERE clause, I can easily find all rows for the state of New York.

DQMATCH



The DQMATCH function applies a *match-definition* to generate a match code for the *source-text*. The two required arguments are:

- *source-text*, which specifies the input text and
- m*atch-definition*, which identifies the QKB match definition used to produce the match code.

And, as usual, you can specify a *locale* if desired.

In this example, the **sasdata.examples** table contains three rows. Because the name values are not entered consistently, it would be very difficult to identify that they represent the same person. I'll use the DQMATCH function to generate match codes for the **Contact** data. The match codes make it easy to see that these values are all equivalent.

DQEXTRACT

DQEXTRACT(*source-text*, '*extractions-definition*' <, *locale*> **);**

```
data tokens;
   infile "&path/data/rawinfo.txt" _infile_=rawText obs=3;
   input;
   Tokens=dqextract(rawText, 'Contact Info', 'ENUSA');
run;
```

```
Q.Center@example.Com,queen of angels medical
center,279.648.0144,Ashburn, VA
7049354011,albemarle, me,r.aaron@example.com,RONALD AARON
8284565851,waynesville, nc,g.abbate@example.com,GUY ABBATE
```

Tokens
/=/queen of angels medical center/=/Ashburn, VA/=/Q.Center@example.Com/=/279.648.0144/=/,; ,; ,
RONALD AARON/=//=/albemarle, me/=/r.aaron@example.com/=/7049354011/=/,; ,; ,
GUY ABBATE/=//=/waynesville, nc/=/g.abbate@example.com/=/8284565851/=/,; ,; ,

The DQEXTRACT function applies an *extraction-definition* to generate tokens from the *source-text*. The two required arguments are:

- *source-text*, which specifies the input text and
- *extraction-definition*, which identifies the QKB extraction definition used to produce the tokens.

And, as usual, you can specify a *locale* if desired.

In this example, the **rawinfo.txt** file contains rows of messy text. Because the values are not entered in a consistent pattern, it would be very difficult to extract these values reliably. Here I use the DQEXTRACT function to extract the values into tokens based ont eht **Contact Info** extraction definition. This produces reliably extracted values in a delimited token string.

DQEXTTOKENGET

DQEXTTOKENGET(*delimited-string ,token ,extraction-definition' <, locale>***);**



```
data tokens;
   infile "&path/data/rawinfo.txt" _infile_=rawText obs=3;
   input;
   Tokens=dqextract(rawText, 'Contact Info', 'ENUSA');
   Name=dqExtTokenGet(tokens,'Name','Contact Info');
   Org=dqExtTokenGet(tokens,'Organization','Contact Info');
   Email=dqExtTokenGet(tokens,'E-mail','Contact Info');
run;
```

Name	Org	Email
	queen of angels medical center	Q.Center@example.Com
RONALD AARON		r.aaron@example.com
GUY ABBATE		g.abbate@example.com

DEMONSTRATIONS

DEMO 1 – DQ ON THE COMPUTE SERVER

```
/* Identify existing duplicates */
title "Duplicates in the sasdata.prospects";
proc FedSQL number;
select Contact||','||Address||','||City||','||State as Entity
      ,Count(*) as Count
    from sasdata.prospects
    group by 1
    having Count(*)  >1
;
quit;
/* %dqputloc(ENUSA,PARSEDEFN=0,short=1) */
/* Standardize the existing data */
data sasdata.prospects_std;
    set sasdata.prospects;
    select (dqidentify(Contact, 'Field Content')) ;
          when ('INDIVIDUAL') contact=dqstandardize(Contact,'Name') ;
          when ('ORGANIZATION') contact=dqcase(Contact,'Proper
(Organization)') ;
          otherwise contact=propcase(contact) ;
     end;
     Address=dqStandardize(Address,'Address');
     City=dqstandardize(City,'City');
     State=dqstandardize(State,'State/Province (Abbreviation)');
     Phone=dqstandardize(Phone,'Phone');
;
run;

/* Identify existing duplicates */
title "Duplicates in standardized data (sasdata.prospects_std)";
proc FedSQL number;
select Contact||','||Address||','||City||','||State as Entity
      ,Count(*) as Count
    from sasdata.prospects_std
    group by 1
    having Count(*)  >1
;
quit;

/* But what about John Q. Smith?? */
title "John Q. Smith duplication not detected";
proc FedSQL;
select ContactID,Contact,Address,City,State
    from sasdata.prospects_std
    where ContactID between 970 and 972
;
quit;

/* Behold - the power of match codes! */
title "Match codes could detect this duplication";
proc FedSQL;
select Contact,dqmatch(Contact,'Name')
      ,Address,dqmatch(Address,'Address')
      ,City,dqmatch(City,'City')
      ,State,dqmatch(State,'State/Province')
```

```
    from sasdata.prospects_std
    where ContactID between 970 and 972
;
quit;

/* Ok - let's fix this data! */

/* Copy out the duplicates */
proc FedSQL ;
drop table duplicates FORCE;
create table duplicates as
/* Construct ClusterID by concatenating match codes for Name, Addressm,
City, and State */
select
dqmatch(Contact,'Name')||dqmatch(Address,'Address')||dqmatch(City,'City')||
dqmatch(State,'State/Province') as ClusterID
      ,*
    from sasdata.prospects_std
    where
dqmatch(Contact,'Name')||dqmatch(Address,'Address')||dqmatch(City,'City')||
dqmatch(State,'State/Province')
          in (select
dqmatch(Contact,'Name')||dqmatch(Address,'Address')||dqmatch(City,'City')||
dqmatch(State,'State/Province') as Entity
                      from sasdata.prospects_std
                      group by 1
                      having Count(*)  >1)
    /* Sort into clusters, with the most complete record last in the
cluster */
    order by ClusterID, Contact, State, City, ZIP, Address,
HouseholdIncome, Phone;
quit;

/* Keep the last record in each cluster as the surviving record */
data survivors (drop=clusterID);
    set duplicates;
    by clusterID;
    if last.clusterID then output;
run;

/* Remove all duplicates from the orignal data */
proc FedSQL;
delete from sasdata.prospects_std
    where ContactID
          in (select ContactID
                      from duplicates);
quit;

/* Then append the duplicate survivors */
proc append base=sasdata.prospects_std data=survivors;
run;

proc sort data=sasdata.prospects_std;
    by ContactID;
run;

/* Test to see if any of the problem records still have duplicates in the
final data */
```

```
title "No two names are alike here, so duplicate records no longer exist";
proc FedSQL;
select *
    from sasdata.prospects_std
    where ContactID
          in (select ContactID
                      from duplicates);
quit;
title;

/* Using PROC DQMATCH for Match Codes and Clustering */
/* Run the DQMATCH procedure. */
proc dqmatch data=sasdata.prospects_std out=sasdata.prospects_matchcoded
matchcode=match_code
   cluster=cluster_group locale='ENUSA';
   criteria matchdef='Name' var=Contact;
   criteria matchdef='Address' var=Address;
   criteria matchdef='City' var=City;
   criteria matchdef='State/Province' var=State;
run;

/* Print the results. */
title 'Using PROC DQMATCH for Name and Address match clustering';
title2 'at the default sensitivity';
proc print data=sasdata.prospects_matchcoded(obs=8);
    var ContactID CLUSTER_GROUP Contact Address     City  State MATCH_CODE;
run;
```

DEMO 2 – DATA QUALITY IN CAS

```
/*************************************************************************
****
 - SASDATA and WORK librefs replaced with CASUSER
 - PROC FedSQL uses the sessref= option & "number" is no longer effective.
*************************************************************************
***/
/* Identify existing duplicates */
title "Duplicates in the casuser.prospects";
proc FedSQL sessref=conn number;
select Contact||','||Address||','||City||','||State as Entity
      ,Count(*) as Count
    from casuser.prospects
    group by 1
    having Count(*)  >1
;
quit;
/* %dqputloc(ENUSA,PARSEDEFN=0,short=1) */
/* Standardize the existing data */
data casuser.prospects_std;
    set casuser.prospects;
    select (dqidentify(Contact, 'Field Content')) ;
          when ('INDIVIDUAL') contact=dqstandardize(Contact,'Name') ;
          when ('ORGANIZATION') contact=dqcase(Contact,'Proper
(Organization)') ;
          otherwise contact=propcase(contact) ;
     end;
     Address=dqStandardize(Address,'Address');
     City=dqstandardize(City,'City');
     State=dqstandardize(State,'State/Province (Abbreviation)');
     Phone=dqstandardize(Phone,'Phone');
;
run;

/* Identify existing duplicates */
title "Duplicates in standardized data (casuser.prospects_std)";
proc FedSQL sessref=conn  number;
select Contact||','||Address||','||City||','||State as Entity
      ,Count(*) as Count
    from casuser.prospects_std
    group by 1
    having Count(*)  >1
;
quit;

/* But what about John Q. Smith?? */
title "John Q. Smith duplication not detected";
proc FedSQL sessref=conn ;
select ContactID,Contact,Address,City,State
    from casuser.prospects_std
    where ContactID between 970 and 972
;
quit;

/* Behold - the power of match codes works in CAS, too! */
title "Match codes could detect this duplication in CAS, too!";
proc FedSQL sessref=conn ;
select Contact,dqmatch(Contact,'Name')
```

```
      ,Address,dqmatch(Address,'Address')
      ,City,dqmatch(City,'City')
      ,State,dqmatch(State,'State/Province')
    from casuser.prospects_std
    where ContactID between 970 and 972
;
quit;

/* Ok - let's fix this data! */

/* Construct ClusterID by concatenating match codes for Name, Addressm,
City, and State */
proc FedSQL sessref=conn;
drop table casuser.clusterIDs force;
create table casuser.clusterIDs as
    select
dqmatch(Contact,'Name')||dqmatch(Address,'Address')||dqmatch(City,'City')||
dqmatch(State,'State/Province') as clusterID
          ,count(*) as Counts
         from casuser.prospects_std
         group by 1
;
quit;

proc FedSQL sessref=conn;
drop table casuser.duplicates force;
create table casuser.duplicates as
    select p.*, ClusterID,counts
          from casuser.prospects_std as p
            inner join
                  casuser.clusterIDs as c
               on
c.clusterID=dqmatch(Contact,'Name')||dqmatch(Address,'Address')||dqmatch(Ci
ty,'City')||dqmatch(State,'State/Province')
;
quit;

/* Pick the last record in the cluster as the surviving record */
data casuser.prospects_final (drop=clusterID counts) /sessref=conn;
    set casuser.duplicates;
    by ClusterID Contact State City ZIP Address HouseholdIncome Phone;
    if last.clusterID then output;
run;

/* Check out the results */
proc FedSQL sessref=conn ;
select p.*
    from casuser.prospects_final as p
    inner join
    casuser.duplicates as d
    on p.ContactID=d.ContactID
    where counts>1
    order by p.contact, p.address, p.city, p.state
;
quit;
```

PRACTICES

PRACTICE 1 – IDENTIFY AND CASE

1. In the practice folder, open the program file **Practice 1 - Identify and case.sas**
2. Run the code in the "Setup and Exploration" section.
3. Review the results.
4. Modify the PROC FEDSQL step in the "Solve the Problem" section to use the DQCASE function to correct the casing of the data in the **Text** column.
5. Run your modified program code. Expected result:

Corrected Data

Text	CORRECTEDTEXT
MR. JOHN DOE	Mr. John Doe
SAS INSITUTE, INC.	SAS Insitute, Inc.
123 MCLAWS CIRCLE	123 Mclaws Circle

PRACTICE 2: FUZZY MATCHING COMPLEX PROBLEM

Background

We work for a company who makes gourmet cat food for discerning cat lovers. To expand our market base, we've purchased the mailing list from Cat Lovers magazine. We want to first exclude anyone on the list who is already our customer, then mail the remaining cat lovers a sampler pack of our cat food to entice them to become customers.

Our customer data contains extensive information including the customer's name and address, all properly formatted. The mailing list contains only names an addresses, formatted specifically for mailing. Names an addresses may not be entered the same way and in the same format as in our data. There is no unique ID number available in both data sets for allow a simple join. We're going to need to do some fuzzy matching here…

Instructions

1. In the practice folder, open the program file **Practice 2 - Fuzzy matching complex problem.sas**
2. Run the code in the **Setup & Exploration** section and review the results.
    a. Note the differences in the formatting of the values in the Customers database and Cat Lovers mailing list.
    b. Note that after traditional standardization, the join indicates only 3 customers are cat lovers.
3. Using the starter code provided in the **Solve the Problem** section and the instructions provided in the comments, generate match codes and use them to exclude current customers from the solicitation mailing list.

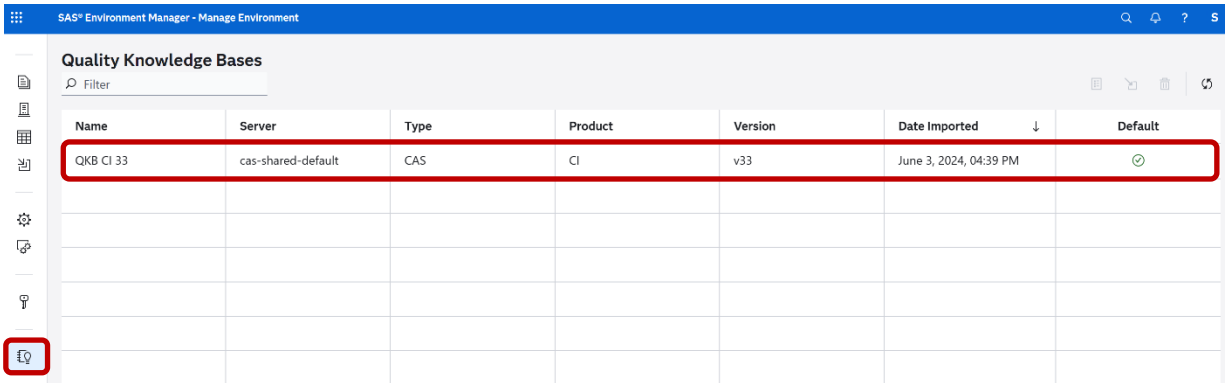
PRACTICE 3 – EXTRACT DATA FROM TEXT

1. In the practice folder, open the program file **Practice 3 – Extract Data from Messy Text.sas**
2. Run the code in the "Setup & exploration" section and review the results. Note that in this '|' delimited file, some records include company information as the second item, and some do not - so not all record layouts are the same.
3. Modify the starter code to in the Solve the Problem section to extract the data from each record and standardize the text in the results.

EXPLORATION SOLUTIONS

EXPLORE 1 – WHICH QKB IS AVAILABLE?

Step 4: QKB CI 33 is installed



Step 6: Is QKB CI 33 available? **YES**

```
QKBs in repository: QKB CI 33
```

Step 7: What is the default locale for this session? **ENUSA**

```
Group=DATAQUALITY
DQLOCALE=(ENUSA)
    Specifies the Data Quality Server ordered list of locales for data
    cleansing.
DQQKBREPOLOC=!SASHOME/share/refdata/qkb
    Specifies the location of the Quality Knowledge Base Repository.
DQSETUPLOC=QKB CI 33
    Specifies the location of the Quality Knowledge Base root directory.
```

EXPLORE 2 – INVESTIGATING LOCALES

1. Run step 1. What locales are loaded for this session?
   ENUSA
2. Run the code in Step 2 to load an additional locale, then re-run the code in step 1. What locales are loaded now?
   ENUSA, PTBRA
3. Run the code in Step 3.
   How many case definitions are available for the ENUSA locale? **11**
   How many case definitions are available for the PTBRA locale? **4**

PRACTICE SOLUTIONS

PRACTICE 1 – IDENTIFY AND CASE

```
/*****************************************************************
 Solve the problem:
 Modify the starter code to use the dqcase function to correct
 the casing of Text
*****************************************************************/
title "Corrected Data";
proc FedSQL;
select Text
      ,case (dqidentify(text,'Field Content'))
         /* Add your code here */
          when 'INDIVIDUAL' then dqcase(Text, 'Proper (Name)')
          when 'ORGANIZATION' then dqcase(Text, 'Proper (Organization)')
          when 'DELIVERY ADDRESS' then
                dqcase(Text, 'Proper (Address Number)')
          else propcase(Text)
       end as CorrectedText
    from sasdq.practice1
;
quit;
title;
```

PRACTICE 2: FUZZY MATCHING COMPLEX PROBLEM

```
/*************************************************************
 Modifed Step 2 code:
 Because some of the mailing list names are cat names, we will
 concatenate the address fields, generate match codes using the
 'City - State/Province - Postal Code' definition to find matches,
 and base our join criteria on the match codes.
 Modify the "Customers Who Are Also Cat Lovers" report to join
 based on those match codes.
*************************************************************/
title "Customers Who Are Also Cat Lovers";
footnote "Note rows 2 and 6. Catherine Thomas and James Attwood";
footnote2 "subscribed to Cat Lovers magazine in their cat's names.";
proc sql number;
select cust.CustomerID
      ,Cat.Name as CatLover
      ,Cust.Name 'CustName'
      ,cat.Address as CatStreet
      ,cust.Street1 'CustStreet'
      ,cat.City as CatCity
      ,cust.City 'CustCity'
      ,Cat.State as CatState
      ,Cust.State 'CustState'
      ,Cat.Zip as CatZip
      ,Cust.Zip 'CustZip'
    from sasdq.Customers as cust
        inner join
         sasdq.CatLovers as cat
    on  dqmatch(catx(',',cat.Address,cat.City,cat.State,cat.Zip)
          ,'City - State/Province - Postal Code')
       =dqmatch(catx(',',cust.Street1,cust.City,cust.State,cust.Zip)
          ,'City - State/Province - Postal Code')
    order by cat.Name
;
quit;
title;footnote;

/*************************************************************
 Modified Step 3:
 Now, modify the SQL to select only Name from the Cat Lovers
 mailing list and write them into a macro variable named ExcludeMe
 separated by the string "," Suppress the SQL report using the
 PROC SQL NOPRINT option.
 The PROC SQL INTO clause is documented here:
 https://documentation.sas.com/doc/en/pgmsascdc/default/sqlproc/n1tupenuhmu
1j0n19d3curl9igt4.htm
*************************************************************/
proc sql noprint;
select Cat.Name
    /* add your INTO clause here. */
    into :ExcludeMe separated by '","'
    from sasdq.Customers as cust
        inner join
         sasdq.CatLovers as cat
```

```
/* Copy your match code from step 2 here */
    on  dqmatch(catx(',',cat.Address,cat.City,cat.State,cat.Zip)
          ,'City - State/Province - Postal Code')
       =dqmatch(catx(',',cust.Street1,cust.City,cust.State,cust.Zip)
          ,'City - State/Province - Postal Code')
;
quit;
%put NOTE: Exclude these &sqlobs names: "%superq(ExcludeMe)";

/*************************************************************
 Step 4:
 Create the solicitation mailing list by selecting all rows from
 sasdq.CatLovers with Name values NOT are in the ExcludeMe list.
*************************************************************/
proc sql;
create table SolicitationMailingList as
select *
    from sasdq.CatLovers
    /* Write your WHERE clause here. Don't forget to resolve ExcludeMe
inside double quotes!*/
    where Name not in ("&ExcludeMe")
;
quit;

/*************************************************************
 Step 5:
 Run the solicitation mailing list report step below. Review the
 report and answer the question.

*************************************************************/

title "Solicitation Mailing List";
proc sql number;
select *
    from SolicitationMailingList
    order by Name
;
quit;
title;

/*************************************************************
Question:
 Which row numbers appear to have subscription in the cat's name
 instead of the cat owner's?

 Answer:
 Rows 10 and 11
*************************************************************/
```

PRACTICE 3 – EXTRACT DATA FROM TEXT

```
/*****************************************************************
 Solve the problem:
 Modify the starter code to:
   a. Tokenize the data in the text file using dqExtract
   b. Extract the Name, Organization, Address, Email, and Phone date
      from the tokens generated by dqExtract using dqExtTokenGet
   c. Standardize the name, address, and phone data from dqExtTokenGet
      using dqStandardize
   d. Standardize the Organization data from dqExtTokenGet using dqCase.
   e. Standardize the Email data from dqExtTokenGet using the SAS
      lowcase function.

Save the results to sasdq.NewCustomers
*****************************************************************/

data sasdq.NewCustomers;
    infile "&path/data/rawinfo2.txt";
    input;
    /* Extract the tokens from the raw data only once for efficiency */
    _tokens=dqextract(_infile_,'Contact Info','ENUSA');
    drop _tokens:;
    /* Get each data value and standardize the text */
    Name=dqstandardize(dqExtTokenGet(_tokens
                ,'Name', 'Contact Info', 'ENUSA'),'Name');
    Organization=dqcase(dqExtTokenGet(_tokens
                ,'Organization', 'Contact Info', 'ENUSA')
                ,'Proper (Organization)');
    Address=dqStandardize(dqExtTokenGet(_tokens
                ,'Address', 'Contact Info', 'ENUSA')
                ,'City - State/Province - Postal Code');
    Email=lowcase(dqExtTokenGet(_tokens
                ,'E-mail', 'Contact Info', 'ENUSA'));
    Phone=dqstandardize(dqExtTokenGet(_tokens
                ,'Phone', 'Contact Info', 'ENUSA'),'Phone');
run;

title "Practice 3 - New Customer Data";
proc print;
run;
title;
```

ADDITIONAL RESOURCES

SAS COMMUNITIES LIBRARY ARTICLES

- Coding for Data Quality in SAS Viya Part 1 – Fundamentals
- Coding for Data Quality in SAS Viya Part 2 – Standardization
- Coding for Data Quality in SAS Viya Part 3 – Fuzzy Matching with Match Codes
- Coding for Data Quality in SAS Viya Part 4 – Extracting Data from Messy Text Files
- SAS Quality Knowledge Base (QKB) Customization Tools Available for SAS QKB Customers

DOCUMENTATION

- SAS Data Quality Language Reference
- SAS® Data Quality: CAS Action Programming Guide

CONTACT INFORMATION

Your comments and questions are valued and encouraged. Contact the author at:

Mark Jordan
Email: sas.jedi@gmail.com
X: @JediRetired

Download link:

Coding for Cleanliness

https://bit.ly/Tut-Code4Cleanliness

SAS and all other SAS Institute Inc. product or service names are registered trademarks or trademarks of SAS Institute Inc. in the USA and other countries. ® indicates USA registration. Other brand and product names are trademarks of their respective companies.